

Machine Learning Project Documentation

Signal Object Detection

Popescu Iulia-Maria

June 9, 2026

Contents

1	Introduction	3
2	Data Preprocessing and Feature Representation	3
2.1	For the Classical Models (Naive Bayes and SVM)	3
2.2	For the Random Forest Approach (HOG)	3
3	Classical Machine Learning Models	4
3.1	Gaussian Naive Bayes (Kaggle Result: 0.27127)	4
3.2	Support Vector Machine (SVM) (Kaggle Result: 0.25890)	4
3.3	Random Forest on HOG Features (Kaggle Result: 0.35418)	4
4	Convolutional Neural Networks (CNN)	4
4.1	Variant 1: Baseline CNN (Kaggle Result: 0.50545)	5
4.2	Variant 2: Classifier Refinement (Kaggle Result: 0.51418)	5
4.3	Variant 3: Deep CNN (Kaggle Result: 0.63636)	5
4.4	Variant 4: State-of-the-Art Techniques (Kaggle Result: 0.72363)	6
4.5	Variant 5: Fine-Tuning and Class Weights (Kaggle Result: 0.74545)	6
5	Exploring Architectures and Additional Adjustments (Variants 6, 7, and 8)	7
5.1	Variant 6: Penalizing the Majority Class (Score: 0.48363)	7
5.2	Variant 7: Widened Architecture (Score: 0.21890)	7
5.3	Variant 8: Ensemble Calibration and Bootstrapping (Score: 0.74618 - best)	8
6	Maximizing Performance: 5-Fold, Blending, and Pseudo-Labeling	8
6.1	The Fold-Based Approach (Kaggle: 0.75345 - 0.77454)	8
6.2	The Pseudo-Labeling Experiment (An Instructive Failure)	9
6.3	The Final Ensemble: Blending and Additional Seeding	9

7	Results and Performance Analysis	9
7.1	Evolution of Loss and Accuracy — Standard 5-Fold	10
7.2	Accuracy Evolution — 5-Fold Optimized with EMA	11
7.3	Impact of the Learning Rate on Accuracy (Fold 1)	12
7.4	Confusion Matrices — HOG + Random Forest vs. CNN Fine-Tuned	13
7.5	Comparative Summary	14
8	Conclusions	15

1 Introduction

This document details the process of developing, training, and optimizing various Machine Learning models for classifying images into 5 distinct categories (classes). The objective was to build the most robust classifier possible, starting from classical Machine Learning approaches and progressing to deep Convolutional Neural Networks (CNNs), optimized with advanced techniques.

The documentation is structured logically, emphasizing practical implementation, data preprocessing, augmentation techniques, hyperparameter settings, and the comparative results of each experiment, avoiding excessive mathematical derivations and focusing instead on justifying engineering decisions.

2 Data Preprocessing and Feature Representation

A critical component of any Machine Learning pipeline is preprocessing. Depending on the chosen model, the data was handled differently:

2.1 For the Classical Models (Naive Bayes and SVM)

The images were first converted to RGB color space, after which the pixels were flattened into a one-dimensional vector. To cope with the high dimensionality of the features and to improve model performance, we applied the following pipeline:

- **StandardScaler:** The data was standardized so that each feature has a mean of zero and a standard deviation of 1. This step is essential because PCA and SVM are particularly sensitive to the scale of different features.
- **PCA (Principal Component Analysis):** To reduce the huge dimensional space of the flattened images, we projected the data onto the first 200 principal components. The number 200 was chosen empirically as an excellent compromise between retaining significant variance in the images and drastically reducing noise, thereby preventing overfitting.

2.2 For the Random Forest Approach (HOG)

Instead of flattening the raw pixels, the images were converted to grayscale in order to extract HOG (Histogram of Oriented Gradients) descriptors. HOG focuses on the structure and shape of the objects in the image. The hyperparameters chosen for HOG were:

- 9 gradient orientations.

- Blocks of 2×2 cells and cells of 8×8 pixels.
- L2-Hys block normalization, to be invariant to lighting changes.

3 Classical Machine Learning Models

Starting from the preprocessed dataset, we trained 3 classical models to establish a performance baseline:

3.1 Gaussian Naive Bayes (Kaggle Result: 0.27127)

This model uses a Gaussian density assuming independence between the features extracted by PCA. Although it is a very fast and easy model to train, the lack of correlation between features assumed by Naive Bayes does not make it the best fit for images. Nevertheless, it provided a reasonable starting point for validating the preprocessing pipeline.

3.2 Support Vector Machine (SVM) (Kaggle Result: 0.25890)

We used an SVM classifier with an RBF (Radial Basis Function) kernel. This kernel helps the model handle the non-linear separability of the image classes in the PCA-reduced space.

- The regularization parameter C was set to 1.0.
- The hyperparameter γ (gamma) was set to `scale` mode, helping the model automatically adapt to the variance of the training data.

3.3 Random Forest on HOG Features (Kaggle Result: 0.35418)

Using the extracted HOG feature matrix, we trained a Random Forest classifier.

- **Hyperparameters:** We used 300 decision trees ($n_estimators=300$) and a maximum tree depth of 20 ($max_depth=20$). The $n_jobs=-1$ parameter was used for parallel training.
- This method has the advantage of detecting shapes and edges well, reducing the influence of the background.

4 Convolutional Neural Networks (CNN)

The true classification power came from the Deep Learning models. The process of developing the CNNs was iterative. I will describe the progression of the architectures and the improvements made at each stage.

4.1 Variant 1: Baseline CNN (Kaggle Result: 0.50545)

The first version introduced an `ImprovedResidualCNN` class.

- **Architecture:** 3 convolutional layers. A residual (shortcut) connection was used for stability, along with Batch Normalization and ReLU activation. The final pooling was done through an `AdaptiveAvgPool2d(7x7)` layer, followed by Dropout (0.5) and a Fully Connected (Linear) layer with output equal to the number of classes (5).
- **Augmentation:** Minimal, including `RandomVerticalFlip` and `RandomErasing(p=0.25)`, together with normalization to $[-1, 1]$.
- **Training:** AdamW optimizer (LR=3e-4) combined with a `CosineAnnealingLR` scheduler for 20 epochs.

4.2 Variant 2: Classifier Refinement (Kaggle Result: 0.51418)

- **Changes:** The feature extraction architecture remained similar, but the final classification layer was split into two fully connected layers (`fc1=128` and `fc2=5`), separated by a dropout layer and an additional ReLU layer.
- **Augmentation:** A random rotation was added to the preprocessing (`RandomRotation(15)`). The model was trained for 30 epochs, bringing a slight improvement in accuracy.

4.3 Variant 3: Deep CNN (Kaggle Result: 0.63636)

Bringing a significant performance boost, this variant introduced major structural changes:

- **Deepened Architecture:** We gradually increased the number of channels up to 128 (5 Conv2d layers), split into 3 main blocks, each block assisted by residual connections (shortcuts).
- **SiLU Activation:** The ReLU function was completely replaced by SiLU, a smoother activation function that typically improves convergence and regularization in deep networks.
- **Aggressive Dropout:** We raised the dropout rate to 0.6 for the final layer.
- **Training and Optimizer:** The number of epochs increased to 60. Gradient clipping (at 1.0) was implemented for the first time to stabilize training. We also tested an alternative scheduler (`SequentialLR`) consisting of a `LinearLR` for warm-up during the first 10 epochs, followed by `CosineAnnealingLR`.

4.4 Variant 4: State-of-the-Art Techniques (Kaggle Result: 0.72363)

This model was based on a ResNet-like architecture (`SignalResNet`), but adapted and heavily regularized through State-of-the-Art techniques:

- **Noise and Complex Augmentation:** The transformations became much more aggressive: `RandomAffine`, `RandomRotation(15)`, a custom `GaussianNoise` applied to the tensor, plus `RandomErasing` on random regions. This massive variation forces the model not to memorize the data.
- **MixUp and CutMix:** Implemented jointly (50%-50% choice). These methods combine two images and their labels simultaneously (through transparency interpolation for MixUp, or crop copying for CutMix), forcing the network to produce smoother confidence predictions.
- **Label Smoothing (0.1):** Instead of forcing the model to predict 100% for one class and 0% for the rest, label smoothing reduces overconfidence and penalizes the model for rigidity.
- **EMA (Exponential Moving Average):** In parallel with training, a second set of “weights” (EMA) was updated as an exponentially weighted average of the current model (`decay=0.999`). The EMA model tends to generalize better on the unseen test set.
- **Submission Generation via TTA (Test-Time Augmentation):** At inference time, the final predictions were obtained by passing each image through 4 variants (Normal, Horizontal Flip, Vertical Flip, Horizontal+Vertical Flip) and averaging the Softmax probabilities. Furthermore, a **simple ensemble** was created by averaging the probabilities of the base model and the EMA model.

4.5 Variant 5: Fine-Tuning and Class Weights (Kaggle Result: 0.74545)

This represented the culmination of the project, focusing on refining the previous model (V4).

- **Identifying Class Bias:** Through a *value counts* analysis, we observed that the network had developed a strong bias toward class 1. To correct this, **Class Weights** were computed based on the inverse frequency of classes in the training data, manually adding a “boost” for the non-1 classes. This weight vector was passed directly to the `CrossEntropyLoss` function.

- **Reducing Augmentations:** Since the model was already well generalized (via V4), the extreme augmentations (`RandomErasing`) were removed to allow the model to perceive fine details during the fine-tuning stage.
- **Low Learning Rate:** Training was done with a very small base LR (5×10^{-5}) for 35 epochs, allowing the model to settle optimally into the global minimum.
- **Extended Ensemble:** In the end, the predictions were generated using an ensemble of 4 models: `best_model_v4`, `best_model_ema_v4`, `fine_tuned_model`, `fine_tuned_model_ema`. The fine-tuned models were given a higher weight (1.2 versus 1.0) when summing the logits.

5 Exploring Architectures and Additional Adjustments (Variants 6, 7, and 8)

In an attempt to explore the limits of performance and mitigate overfitting, we continued with several architectural and regularization changes.

5.1 Variant 6: Penalizing the Majority Class (Score: 0.48363)

This experiment was based on a manual rebalancing approach.

- We defined strict class weights (`CLASS_WEIGHT_VALUES = [0.60, 1.10, 1.10, 1.10, 1.10]`) to discourage the model from over-predicting class 1.
- The number of epochs was increased to 150 to allow slower convergence with Label Smoothing of 0.08.
- **Diversity through Seeding:** We ran training with several different seeds (`ALL_SEEDS = [42, 43, 44, 45, 46]`) to obtain greater variance, then combined the predictions generated by these models at the end.
- Although theoretically valid, this fixed penalization led to a drastic drop in overall performance (0.483), demonstrating that the model needs finer calibration than static weights.

5.2 Variant 7: Widened Architecture (Score: 0.21890)

Here we aimed to increase the model’s capacity to retain information by massively increasing the number of parameters.

- **Architectural changes:** The base channels were doubled (`BASE_CHANNELS = 128`), and the number of residual blocks per stage was increased to 3.

- **Regularization:** To counteract the explosion in the number of parameters, we increased the DROPOUT rate to 0.4.
- **Result:** The increased complexity without additional data led to very poor performance (0.218), suggesting a strong overfitting phenomenon.

5.3 Variant 8: Ensemble Calibration and Bootstrapping (Score: 0.74618 - best)

After the failure of variants 6 and 7, we returned to a stable model (64 channels) and focused on how the final predictions were generated.

- **Diversity through Seeding:** The model was trained on different validation partitions (by changing the seed). This method ensured greater variance among the models, crucial for a strong ensemble.
- **Post-Training Calibration (`kaggle_var8_calibrate`):** We tested different result-weighting strategies (e.g., giving more importance to the EMA models) to optimize the final prediction on the validation set, achieving an improvement to 0.746.

6 Maximizing Performance: 5-Fold, Blending, and Pseudo-Labeling

After stabilizing the optimal architecture, the focus shifted to the validation and training structure in order to surpass a score of 0.75.

6.1 The Fold-Based Approach (Kaggle: 0.75345 - 0.77454)

Instead of a single Train/Validation split, we moved to a **Stratified 5-Fold Cross-Validation** strategy.

- The training set was split into 5 "slices" (folds). Five independent models were trained, each validated on a different portion of the data. The final predictions were the **average of the logits** across all 5 folds.
- **Initial Variant (0.75345):** Standard training on 5 folds.
- **Optimized Variant (`5fold_improved` - 0.77454):** Mixed precision training (`autocast`) was implemented, reducing training time and allowing larger batches, achieving the peak score. Starting with this variant, images were processed exclusively through the *green channel* of the RGB space, as it was empirically observed to contain the highest informational contrast for the signals in this dataset.

- **High-Resolution Variant (5fold_hires - 0.77236):** We increased the input image resolution from 128×64 to 160×96 , also scaling the SpecAugment augmentation proportionally. The idea was that the higher resolution would allow capturing finer details of the signals.
- **Blending (blend_5fold):** The blending script logically combined the two main sources: the probabilities from the base model (128×64) and from the High-Resolution model (160×96). This improved the final predictions.

6.2 The Pseudo-Labeling Experiment (An Instructive Failure)

We implemented **Pseudo-Labeling** using the good predictions generated previously. The concept involves assigning labels to test examples the model is highly confident about (> 0.90 confidence) and using them in training.

- **Result/Observation:** Although theoretically sound, the experiment failed. It was discovered that the model was overconfident only about class 1 (already over-represented), generating 751 pseudo-labels for class 1, and **ZERO** for the difficult classes (3 and 4).
- This attempt demonstrated that pseudo-labeling without a rebalancing filter only exacerbates existing biases, with the final prediction distribution remaining unchanged and the score not improving.

6.3 The Final Ensemble: Blending and Additional Seeding

The final step for squeezing out every last percentage point of accuracy was generating a mega-ensemble (Ensemble Blending).

- **5fold_improved_seed123_4fold (0.77381):** We repeated 5-Fold training on a similar architecture but limited it to 4 folds for efficiency, and changed the seed to 123. This provided a third independent "brain".
- **Blending (blend_5fold_seed123):** The final blending script logically combined all 3 major sources: the probabilities from the base model (128×64), the High-Resolution model (160×96), and the model with a different seed. The final weighting of these matrices provided massive resistance to variance, making the model much more robust.

7 Results and Performance Analysis

Model performance improved progressively as architectural complexity increased, augmentations were enriched, and the validation strategy was refined. Table 1 summarizes

the scores obtained on the public Kaggle set for each development stage.

Table 1: Summary table of models and scores on the Kaggle Test set

Model / File Name	Key Techniques	Kaggle Score
CNN v1	Baseline ConvNet, minimal augmentations	$\approx 0,505$
CNN v2	+ RandomRotation, + additional FC layer	$\approx 0,514$
CNN v3	5 conv. layers, SiLU, Grad Clip, Dropout 0.6	$\approx 0,636$
CNN v4	MixUp/CutMix, EMA, Label Smoothing, TTA	$\approx 0,723$
CNN v5 (fine-tuned)	Class Weights, attenuated augmentation	$\approx 0,745$
HOG + Random Forest	HOG descriptors, 300 trees	0,354
kaggle_var6_cnn	Aggressive Class Weights	0,483
kaggle_var7_cnn	Widened architecture (128 channels)	0,218
kaggle_var8_calibrate	Ensemble bootstrapping, calibration	0,746
5fold_0.75345	Standard 5-Fold Cross-Validation	$\approx 0,753$
5fold_hires_0.77236	5-Fold at 160×96 resolution	$\approx 0,772$
5fold_improved_seed123_4fold	4-Fold, alternative seed (123)	$\approx 0,773$
5fold_improved_0.77454	5-Fold + Mixed Precision	$\approx 0,774$
blend_5fold_seed123	Weighted blending, 3 sets	$\approx 0,774$

7.1 Evolution of Loss and Accuracy — Standard 5-Fold

Figure 1 illustrates the evolution over 100 epochs of the loss function and accuracy for the first fold of the 5fold_0.75345 variant. It can be observed that:

- **Train Loss** (blue line) decreases monotonically and steadily, a sign that the model is progressively assimilating information from the training set.
- **Validation Loss** (orange) is significantly more volatile, with periodic spikes — typical behavior when the learning rate resets in cosine cycles (CosineAnnealingLR). Each sudden increase corresponds to a scheduler restart.
- **Train Accuracy** (green, dashed) rises uniformly toward $\approx 73\%$.
- **Validation Accuracy** (red, dashed) shows large oscillations, but the overall trend is upward, stabilizing in the 70–75% range after epoch 80. The oscillations reflect the model’s sensitivity to the small validation partition specific to a single fold.

These instabilities in Validation motivated the move to additional stabilization techniques (EMA, Mixed Precision) presented in the improved variant.

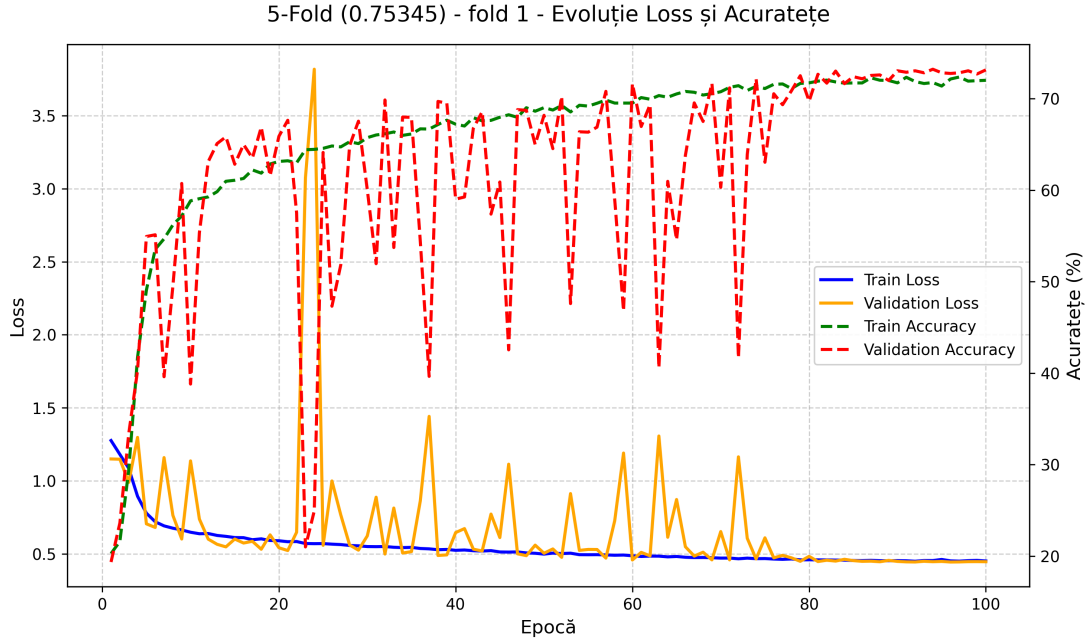


Figure 1: Evolution of Loss and Accuracy — fold 1 of 5fold_0.75345 (overall Kaggle score: 0,753).

7.2 Accuracy Evolution — 5-Fold Optimized with EMA

Figure 2 shows the accuracy evolution for the variant 5fold_improved_0.77454, trained with mixed precision and additional optimizations compared to the standard variant. Three metrics are visualized simultaneously:

- **Train Accuracy** (blue) increases linearly from $\approx 24\%$ to $\approx 67\%$ over 70 epochs, indicating that the model did not reach saturation and could benefit from longer training.
- **Validation Accuracy** (red) starts higher ($\approx 33\%$), rises sharply to 45% at epoch 2, then progresses steadily up to $\approx 75\%$. The fact that validation consistently outperforms training in the early epochs suggests that the aggressive augmentation applied to the training set artificially makes the model's task harder during training, an intentional and positive behavior from a regularization standpoint.
- **EMA Accuracy** (green, dashed) follows the validation trend with a slight delay due to the exponential weight-averaging mechanism, converging to the same level ($\approx 75\%$) by the end of training. It was precisely this stable EMA curve that was used as the criterion for saving the best model (`best_model_ft_ema.pth`).

The absence of violent oscillations — unlike in figure 1 — confirms that Mixed Precision and EMA contributed significantly to stabilizing the training process.

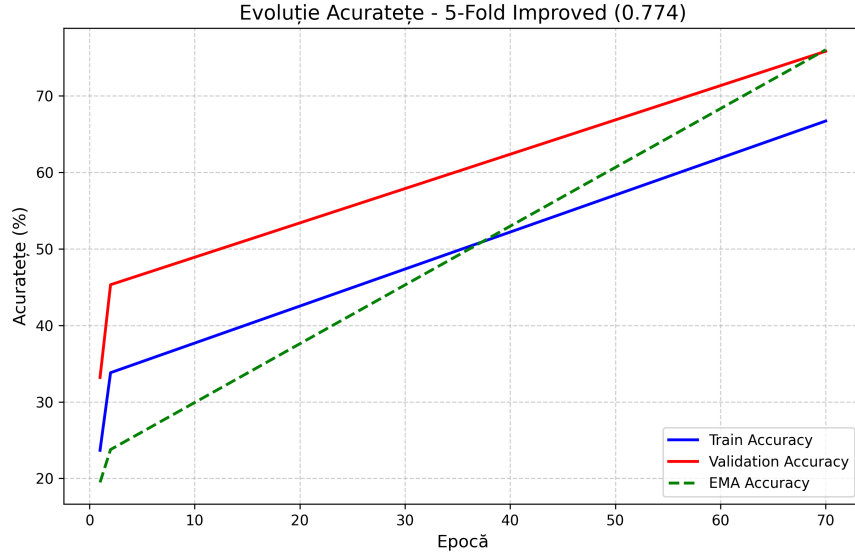


Figure 2: Accuracy Evolution (Train, Validation, EMA) — 5fold_improved_0.77454 (Kaggle score: 0,774).

7.3 Impact of the Learning Rate on Accuracy (Fold 1)

The model 5fold_improved_0.77454 uses a **Warmup Linear + Cosine Annealing** scheduler: the LR increases linearly over the first `WARMUP_EPOCHS=5` epochs from 0 to the maximum value $LR = 1,5 \times 10^{-3}$, after which it decreases along a cosine curve down to $ETA_MIN = 10^{-5}$.

Table 2 presents the LR values at representative epochs and the corresponding accuracies recorded on Fold 1, clearly showing that the progressive reduction of the LR leads to consistent improvements in EMA accuracy, confirming the benefit of the scheduling strategy over a fixed LR.

Table 2: Evolution of the Learning Rate and Validation Accuracy — Fold 1 (5fold_improved_0.77454)

Epoch	LR	Val Acc (%)	EMA Acc (%)
1	$3,0 \times 10^{-4}$	33,19	19,48
5	$1,5 \times 10^{-3}$	48,16	27,19
10	$1,49 \times 10^{-3}$	60,06	48,48
20	$1,34 \times 10^{-3}$	68,58	68,94
30	$1,05 \times 10^{-3}$	71,26	72,23
40	$7,0 \times 10^{-4}$	72,06	74,03
50	$3,6 \times 10^{-4}$	73,74	75,32
60	$1,1 \times 10^{-4}$	75,45	75,77
70	$1,0 \times 10^{-5}$	75,84	76,03
Best	—	76,13	—

It can be observed that the **warmup** phase (epochs 1–5) is critical: the initially small LR (3×10^{-4}) allows for a stable initialization, after which the increase toward $1,5 \times 10^{-3}$ accelerates convergence. Once the maximum is reached, the cosine decay allows the weights to be refined without oscillations, with EMA Accuracy consistently exceeding Val Accuracy in the second half of training.

7.4 Confusion Matrices — HOG + Random Forest vs. CNN Fine-Tuned

Comparing the two confusion matrices in figures 3 and 4 dramatically illustrates the qualitative leap between the classical approaches and those based on deep neural networks.

HOG + Random Forest — Acc: 33,59%

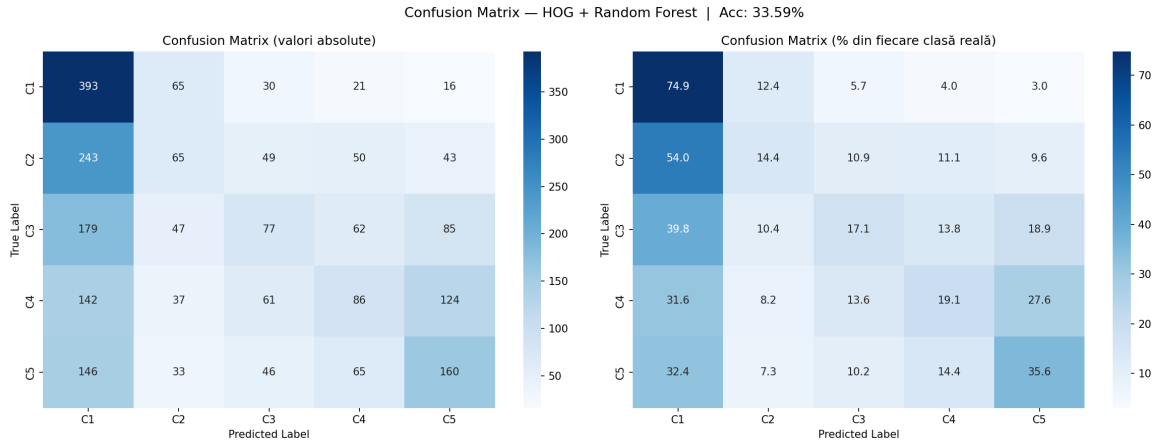


Figure 3: Confusion matrix — HOG + Random Forest (Acc: 33,59%). *Left*: absolute values. *Right*: percentages per true class.

Figure 3 immediately reveals a **severe bias toward class C1**: the model predicts C1 for a massive proportion of all classes. Thus, although C1 reaches a recall of 74,9%, classes C4 and C5 are correctly classified in fewer than 36% of cases, and C3 barely exceeds 17%. This suggests that the HOG descriptors do not sufficiently discriminate the subtle shapes that differentiate classes 3, 4, and 5 from class 1.

CNN Fine-Tuned (best_model_ft_ema.pth) — Acc: 72,84%

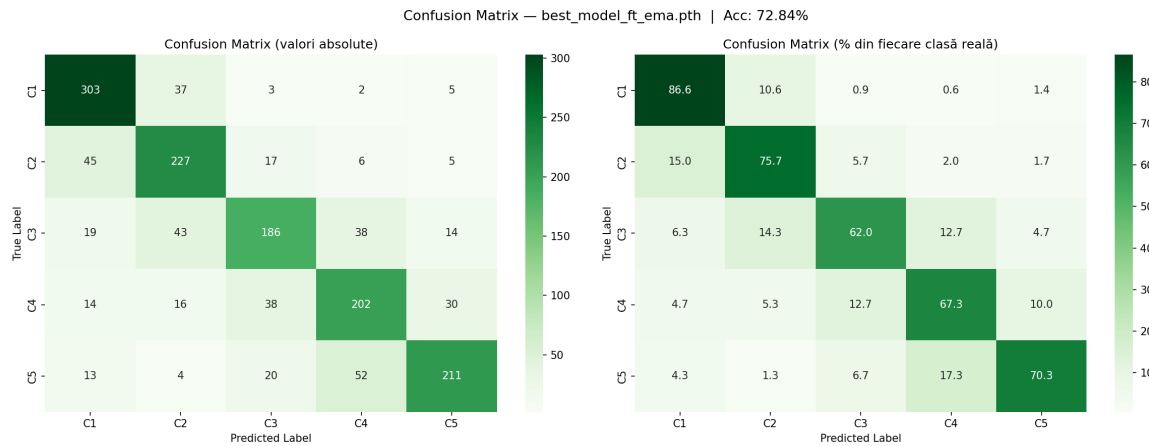


Figure 4: Confusion matrix — CNN Fine-Tuned / best_model_ft_ema.pth (Acc: 72,84%). *Left*: absolute values. *Right*: percentages per true class.

The CNN model shows a clearly dominant main diagonal:

- **C1** — recall 86,6%, the easiest class, with minimal confusion toward C2 (10,6%).
- **C2** — recall 75,7%, with a tendency to be confused with C1 (15,0%), likely due to the spectral similarity between the two categories.
- **C3** — recall 62,0%, the most difficult class; the main confusion is with C2 (14,3%) and C4 (12,7%), reflecting overlapping features.
- **C4** — recall 67,3%, mainly confused with C3 (12,7%) and C5 (10,0%).
- **C5** — recall 70,3%, with confusion toward C4 (17,3%), a consequence of the inter-class proximity at the upper end of the spectrum.

Compared to HOG + Random Forest, the CNN reduces the classification error by over **39** percentage points and completely eliminates the bias toward class C1.

7.5 Comparative Summary

Table 3 comparatively summarizes the per-class accuracy for the two representative models, highlighting the gain brought by the deep learning approach:

Table 3: Per-class accuracy: HOG + RF vs. CNN Fine-Tuned

Class	HOG + RF (recall %)	CNN FT (recall %)
C1	74,9	86,6
C2	14,4	75,7
C3	17,1	62,0
C4	19,1	67,3
C5	35,6	70,3
Overall (Acc)	33,59%	72,84%

8 Conclusions

The iterative experimentation process described in this documentation clearly demonstrated that performance in signal image classification is determined by a balanced combination of **model architecture**, **augmentation regime**, and **validation and ensembling strategy**.

The classical approaches (Naive Bayes, SVM with PCA, Random Forest with HOG) quickly hit a ceiling of $\approx 35\%$, unable to capture the complex spatial relationships between the visual signal features. The confusion matrix of the HOG + Random Forest model (figure 3) clearly highlighted the *class bias* phenomenon — almost all samples were classified as C1, regardless of the true class.

The CNNs immediately surpassed these limits ($\approx 0,505$ for the first variant) and evolved steadily as regularization techniques were added. The most significant gains came from:

- **MixUp / CutMix and Label Smoothing** — reduced overfitting and improved probability calibration ($0,636 \rightarrow 0,723$).
- **Exponential Moving Average (EMA)** on the weights — stabilized training and provided a more robust model for inference ($0,723 \rightarrow 0,745$).
- **Stratified 5-Fold Cross-Validation** — eliminated the variance introduced by a single Train/Validation split and allowed the final predictions to be generated by averaging the logits of the 5 models ($0,745 \rightarrow 0,753$).
- **Increased resolution and Mixed Precision** — allowed capturing fine details and reduced training time, contributing to the peak score ($0,753 \rightarrow 0,774$).
- **Blending with varied seeds** — ensured diversity among the combined models, reducing the variance of the final ensemble.

Comparing the evolutions in figures 1 and 2, it can be seen that the optimized variant eliminates the violent oscillations of the validation function, obtaining much more stable accuracy curves — direct proof of the effectiveness of EMA and Mixed Precision.

The **Pseudo-Labeling** experiment, although it did not improve the score, brought a valuable lesson: applying it without a class-rebalancing mechanism only amplifies existing biases, in this case the over-representation of class C1.

The final score of **0.774** (5-Fold Improved / Blending) represents the maximum achieved within this project, reflecting that the potential of the custom architecture was almost fully exploited given the available dataset. Further steps to increase performance could include using pre-trained architectures (transfer learning with EfficientNet or ViT), augmentations specific to the spectral domain, and more sophisticated pseudo-labeling strategies with explicit class rebalancing.

All scores reported in this document represent the accuracy obtained on the public Kaggle leaderboard.